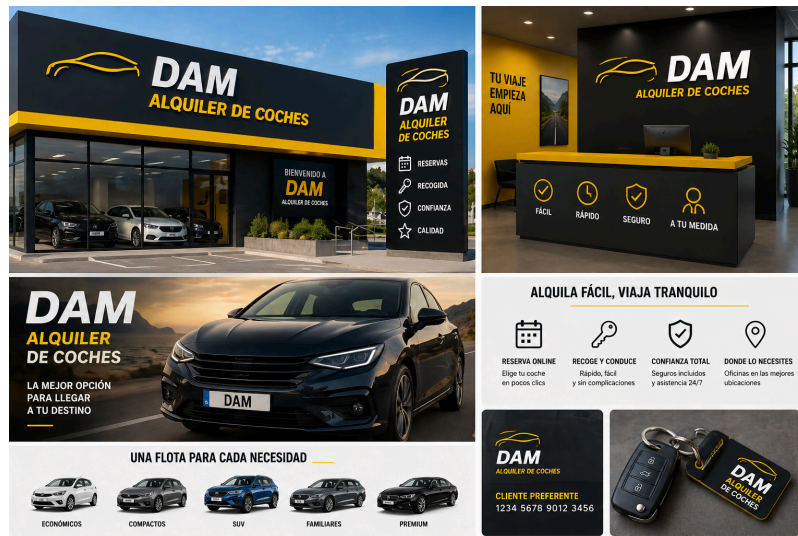


Examen: Gestión de Alquiler de coches



Objetivo

Desarrollar una aplicación Java para gestionar clientes, vehículos y alquileres usando SQLite.

El ejercicio mantiene la misma estructura y complejidad que el proyecto de gimnasio:

- 3 entidades principales.
- 3 servicios.
- Interfaces documentadas con JavaDoc.
- Validaciones con expresiones regulares.
- Herencia.

Qué debes construir

Debes completar la implementación de:

- `ClienteService`
- `VehiculoService`
- `AlquilerService`

Los repositorios SQLite están preparados para trabajar contra la base de datos.

Modelo

Persona

Clase abstracta con:

- `dni`
- `nombre`
- `telefono`
- `email`

Cliente

Debe heredar de **Persona** y añadir:

- **activo**

Vehiculo

- **id**
- **marca**
- **modelo**
- **tipo**
- **disponible**

Tipos permitidos:

- **ECONOMICO**
- **SUV**
- **PREMIUM**

Alquiler

- **id**
- **dniCliente**
- **idVehiculo**
- **fechaInicio**
- **fechaFin**
- **estado**

Estados permitidos:

- **ACTIVO**
- **CANCELADO**
- **FINALIZADO**

Interfaces obligatorias

IClienteService

- **create**
- **findByDni**
- **findAll**
- **update**
- **deleteByDni**
- **findActivos**
- **findByEmail**

IVehiculoService

- **create**
- **findById**

- findAll
- update
- deleteById
- findByTipo

IAlquilerService

- create
- findById
- findAll
- cancelById
- completeById
- findByCliente
- findByVehiculo
- existsActiveRental

Validaciones obligatorias

Implementa `ValidationUtils` con expresiones regulares.

Validación	Regex esperada	Ejemplo válido	Ejemplo no válido
DNI	piensa?	12345678Z	1234A678Z
Email	piensa?	usuario@mail.com	usuario@mail
Teléfono	piensa?	612345678	512345678
Nombre / Marca / Modelo	piensa?	Seat Ibiza	A
Tipo de vehículo	`ECONOMICO	SUV	PREMIUM`
Estado alquiler	`ACTIVO	CANCELADO	FINALIZADO\$`

[illegible]

Reglas de negocio de AlquilerService

- No se puede alquilar si el cliente no existe.
- No se puede alquilar si el cliente está inactivo.

- No se puede alquilar si el vehículo no existe.
- No se puede alquilar si la fecha de inicio es pasada.
- No se puede alquilar si la fecha de fin es anterior o igual a la fecha de inicio.
- No se puede alquilar un vehículo si ya tiene un alquiler activo que se solapa con el rango de fechas solicitado.
- `cancelById` solo puede cancelar alquileres activos.
- `completeById` solo puede finalizar alquileres activos.

Documentación de interfaces

Todas las interfaces deben estar documentadas con JavaDoc.

Cada método debe incluir:

- descripción clara,
- `@param`,
- `@return` si devuelve valor,
- `@throws` si puede fallar o devolver falso por validación.

Ejemplo:

```
/**
 * Crea un alquiler para un cliente y un vehículo.
 *
 * @param alquiler alquiler que se desea crear
 * @return true si el alquiler se ha creado correctamente; false en caso
 contrario
 */
boolean create(Alquiler alquiler);
```

Base de datos de test

Antes de cada test se restaura la base de datos usando `@BeforeEach`.

Nota automática:

```
mvn clean verify -P calificar
```

El informe se genera en:

```
target/nota.txt
```

Pesos

=== RESUMEN FINAL ===

Cliente: 3.00/3.00

Vehiculo: 2.50/2.50

Alquiler: 4.00/4.00

Herencia: 0.50/0.50

Suma por bloques:

$3.00 + 2.50 + 4.00 + 0.50 = 10.00$